

SAE 2.1 — Développement d'une application Java

Démineur

Rapport d'avancement

Membres du binôme

Mohamed OUISSI · Djabrail MOUSTAEV

Année universitaire 2025 – 2026

Table des matières	2
1. Introduction	3
2. Fonctionnalités du programme	4
2.1 Écran de menu principal	4
2.2 Configuration d'une nouvelle partie	4
2.3 Fenêtre de jeu	4
2.4 Interactions avec la grille	4
2.5 Fin de partie	4
2.6 Sauvegarde et reprise	5
3. Structure du programme	6
3.1 Vue d'ensemble	6
3.2 Couche métier	6
Case	6
Grille	6
Partie	6
Sauvegarde	6
3.3 Couche présentation	6
FenetreMenu	6
FenetreConfig	6
FenetreJeu	6
PanneauGrille	6
FenetreFinPartie	7
Main	7
3.4 Diagramme de classes	7
4. Mécanisme de sauvegarde	8
4.1 Principe	8
4.2 Chargement	8
4.3 Déclenchement automatique	8
5. Algorithme de révélation en chaîne	9
5.1 Principe général	9
5.2 Déroulement détaillé	9
5.3 Gestion des marqueurs	9
6. Conclusion	10
6.1 Conclusion de Mohamed Ouissi	10
6.2 Conclusion de Djabrail Moustaeu	10

Table des matières

1. Introduction

Le Démineur est un jeu où il faut réfléchir. Des mines sont cachées dans une grille et le but est de trouver toutes les cases sans tomber sur une mine. Les chiffres aident à savoir où elles sont.

Ce projet a été fait dans le cadre de la SAE 2.01. On devait créer le jeu en Java en binôme, avec Mohamed Ouissi et Djabrail Moustae. On a utilisé Java avec Swing pour faire l'interface graphique, sans utiliser d'autres bibliothèques.

Dans ce rapport, on explique ce qu'on a fait dans le projet, comment le programme est organisé, ainsi que les choix qu'on a faits pour certaines parties. On termine aussi avec une conclusion chacun sur notre expérience.

2. Fonctionnalités du programme

2.1 Écran de menu principal

Au démarrage de l'application, une fenêtre de menu s'affiche et propose trois actions distinctes :

- Nouvelle partie : ouvre une boîte de dialogue de configuration.
- Reprendre la partie : charge la dernière partie sauvegardée (désactivé si aucune sauvegarde n'existe).
- Quitter : ferme l'application.

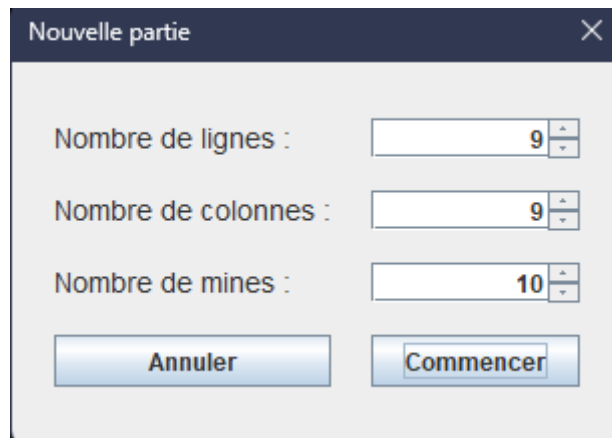


2.2 Configuration d'une nouvelle partie

Avant de lancer une partie, le joueur peut personnaliser trois paramètres via des sélecteurs numériques (spinners) :

- Le nombre de lignes (entre 4 et 30).
- Le nombre de colonnes (entre 4 et 30).
- Le nombre de mines (entre 1 et $\text{lignes} \times \text{colonnes} - 1$).

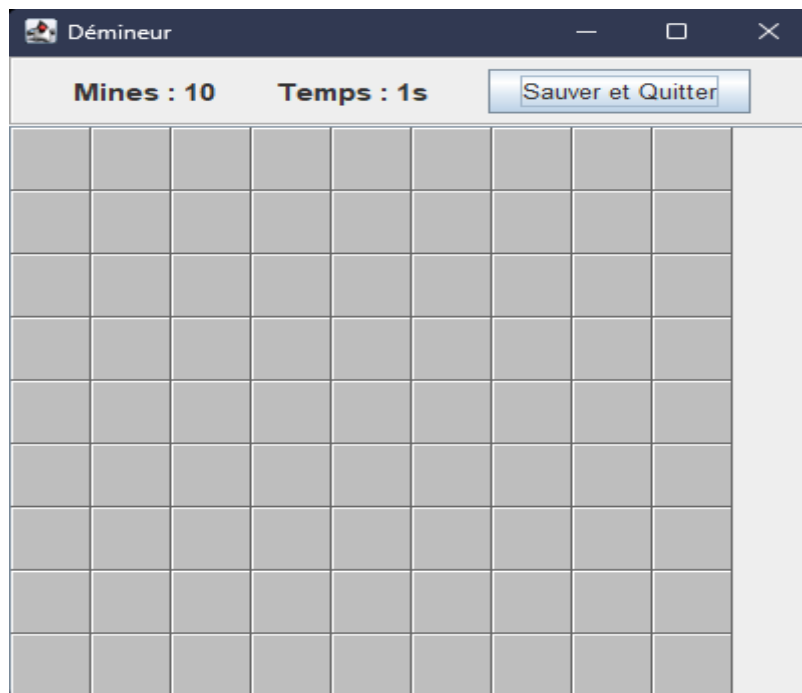
Le maximum de mines est recalculé automatiquement lorsque les dimensions changent, afin de toujours rester cohérent. Les mines sont ensuite placées aléatoirement dans la grille, invisibles pour le joueur.



2.3 Fenêtre de jeu

La fenêtre de jeu affiche plusieurs éléments :

- La grille de jeu sous forme d'un panneau graphique scrollable.
- Un compteur de mines restantes (nombre total de mines moins le nombre de marqueurs étoile ★ posés).
- Un chronomètre indiquant le temps écoulé depuis le début de la partie.
- Un bouton « Sauver et Quitter » permettant de sauvegarder la partie et de revenir au menu.



2.4 Interactions avec la grille

Le joueur interagit avec la grille via la souris :

- Clic gauche sur une case cachée sans marqueur : révèle la case. Si elle est minée, la partie est perdue. Sinon, la case devient visible avec son chiffre d'adjacence. Si ce chiffre est 0, la révélation se propage automatiquement aux voisines (voir section 5).
- Clic gauche sur une case avec marqueur : aucune action (sécurité anti-fausse manipulation).
- Clic droit sur une case cachée : change le marqueur selon le cycle vide → ★ (certitude) → ? (suspçon) → vide.
- Clic sur une case révélée : aucune action.

2.5 Fin de partie

La partie se termine dans deux cas :

- Victoire : toutes les cases non minées ont été révélées.
- Défaite : le joueur a cliqué sur une case minée.

Dans les deux cas, une boîte de dialogue s'affiche indiquant le résultat et le temps réalisé. En cas de défaite, la grille révèle visuellement : la mine ayant explosé (en rouge vif), les mines non marquées (en orange), et les faux marqueurs ★ posés sur des cases saines (barrés d'une croix rouge). Un bouton « Retour au menu » ramène le joueur au menu principal.





2.6 Sauvegarde et reprise

À tout moment durant une partie, le joueur peut sauvegarder via le bouton dédié ou en fermant la fenêtre. La partie est alors mémorisée sur le disque. Lors du prochain lancement, le bouton « Reprendre la partie » du menu devient actif et permet de charger exactement l'état précédent (grille, marqueurs, temps écoulé).

3. Structure du programme

3.1 Vue d'ensemble

Le programme est organisé en plusieurs classes, séparées en deux parties : une partie pour la logique du jeu et une autre pour l'affichage. Chaque classe est dans un fichier différent, comme demandé dans le projet.

3.2 Couche métier

Case

La classe **Case** représente une case de la grille. Elle contient des informations comme si elle est minée, révélée, son marqueur et le nombre de mines autour.

Grille

La classe **Grille** gère toute la grille du jeu. Elle place les mines, calcule les chiffres, gère les clics et vérifie si le joueur a gagné.

Partie

La classe **Partie** représente une partie complète avec la grille et le temps.

Sauvegarde

La classe **Sauvegarde** permet d'enregistrer et de charger une partie, ainsi que de vérifier si une sauvegarde existe ou de la supprimer.

3.3 Couche présentation

FenetreMenu

Fenêtre principale de l'application (JFrame). Affiche les trois boutons du menu et gère la navigation vers les autres fenêtres. Elle implémente ActionListener.

FenetreConfig

Boîte de dialogue modale (JDialog) permettant au joueur de choisir les paramètres d'une nouvelle partie. Implémente ActionListener et ChangeListener pour recalculer le maximum de mines en temps réel.

FenetreJeu

Fenêtre principale de jeu (JFrame). Héberge le PanneauGrille, affiche le compteur de mines et le chronomètre, gère le minuteur Swing (Timer), et intercepte la fermeture de fenêtre pour sauvegarder automatiquement. Implémente ActionListener et WindowListener.

PanneauGrille

Composant graphique (JPanel) qui dessine la grille case par case dans sa méthode paintComponent. Il capture les clics souris (MouseListener), délègue le traitement logique à la Grille, et notifie FenetreJeu des résultats.

FenetreFinPartie

La sauvegarde repose sur le mécanisme de sérialisation objet natif de Java. Ce mécanisme permet de transformer un objet Java, ainsi que l'intégralité du graphe d'objets qu'il référence, en une suite d'octets pouvant être écrite dans un fichier binaire.

4.1 Principe

Pour pouvoir sauvegarder le jeu, certaines classes doivent être sérialisables. Dans notre projet, les classes **Partie**, **Grille** et **Case** implémentent toutes `Serializable`. Elles ont aussi un `serialVersionUID` pour éviter des problèmes si le code change plus tard.

Quand le joueur sauvegarde une partie, un objet **Partie** est enregistré dans un fichier. Cet objet contient la grille, et la grille contient toutes les cases. Donc en fait, toute la partie est sauvegardée en une seule fois.

Ça permet de garder exactement l'état du jeu, avec les mines, les cases révélées, les marqueurs et le temps.

4.2 Chargement

Au chargement, un `ObjectInputStream` relit le fichier et reconstruit le graphe d'objets en mémoire. L'objet **Partie** restauré est identique à celui qui avait été sauvegardé, avec toutes ses cases dans leur état exact. La sauvegarde est ensuite supprimée du disque pour éviter qu'une même partie puisse être reprise deux fois.

4.3 Déclenchement automatique

La sauvegarde est déclenchée dans deux situations : un clic sur le bouton « Sauver et Quitter », ou une fermeture de la fenêtre via le bouton de la barre de titre (événement `windowClosing` intercepté par `WindowListener`). Dans les deux cas, le minuteur est stoppé avant l'écriture afin que le temps sauvegardé soit cohérent.

5. Algorithme de révélation en chaîne

Quand le joueur clique sur une case qui n'a pas de mine autour (donc avec 0 mine adjacente), le jeu ne s'arrête pas là. À la place, il révèle automatiquement les cases autour.

Ça évite de devoir cliquer une par une sur toutes les cases voisines et rend le jeu plus fluide et plus agréable à jouer.

5.1 Principe général

L'algorithme utilisé est un BFS (parcours en largeur). Il commence à partir de la case cliquée et regarde ses cases voisines.

Ensuite, il continue petit à petit en révélant les cases autour, tant qu'elles n'ont pas de mines à côté.

5.2 Déroulement détaillé

L'algorithme utilise une file (Queue) avec la case de départ.

Ensuite, il fonctionne comme ça :

- On prend une case dans la file.
- Si elle est déjà révélée ou marquée, on l'ignore.
- Sinon, on la révèle.
- Si elle a 0 mine autour, on ajoute ses cases voisines dans la file.
- Si elle a un chiffre (donc des mines autour), on l'affiche mais on ne continue pas à partir d'elle.

L'algorithme s'arrête quand il n'y a plus de cases à traiter dans la file. Au final, ça permet de révéler automatiquement toute la zone vide autour du clic.

5.3 Gestion des marqueurs

Une case marquée avec ★ ou ? n'est jamais révélée automatiquement, même si elle est dans une zone à 0 mine autour.

C'est normal dans le Démineur, et ça permet de ne pas enlever les marqueurs que le joueur a posés volontairement.

6. Conclusion

6.1 Conclusion de Mohamed Ouissi

Ce projet a été très intéressant à travailler. Moi personnellement, j'ai compris beaucoup de choses en Java, surtout sur la partie graphique et comment organiser un programme.

J'ai bien aimé le fait de pouvoir tester directement et voir si ça marche ou non. Ça aide vraiment à comprendre ses erreurs et à progresser. Au début c'était pas toujours facile, mais petit à petit j'ai réussi à mieux m'en sortir.

Voir le jeu fonctionner à la fin fait vraiment plaisir. Je pense que ce projet va m'aider pour la suite.

6.2 Conclusion de Djabrail Moustaeu

Ce projet a été intéressant à faire. Ça m'a permis de mieux comprendre Java en faisant quelque chose de concret comme un jeu.

Au début j'avais un peu de mal, mais en avançant j'ai appris pas mal de choses. Le fait de voir le projet avancer petit à petit était motivant.

Le travail en binôme m'a aussi appris à mieux m'organiser. Au final, je suis content de ce qu'on a fait et ça m'a donné plus confiance pour la suite.